| DS | **Session** | Tree 2 | **Question Information** | Level 1 Challenge 75 |
|---|---|---|---|---|

### Question description

A beautiful *code* of a tree of n nodes is a sequence of n−2 integers that uniquely specifies the structure of the tree.

The code is constructed as follows: As long as there are at least three nodes left, find a leaf with the smallest label, add the label of its only neighbour to the code, and remove the leaf from the tree.

Given a beautiful code of a tree, your task is to construct the original tree.

### Constraints

- $3 \le n \le 2 \cdot 10^5$
- $1 \le a,b \le n$

### Input

The first input line contains an integer n: the number of nodes. The nodes are numbered 1,2,…,n.

The second line contains n−2 integers: the beautiful code.

### Output

Print n−1 lines describing the edges of the tree. Each line has to contain two integers a and b: there is an edge between nodes a and b. You can print the edges in any order.

## Logical Test Cases

### Test Case 1

INPUT (STDIN)

```
5
2 2 4
```

EXPECTED OUTPUT

```
1 2
3 2
2 4
4 5
```

### Test Case 2

INPUT (STDIN)

```
5
3 1 2
```

EXPECTED OUTPUT

```
4 3
3 1
1 2
2 5
```

## Mandatory Test Cases

### Test Case 1

KEYWORD

```
q.pop();
```

## Complexity Test Cases

### Test Case 1

CYCLOMATIC COMPLEXITY

```
3
```

### Test Case 2

TOKEN COUNT

```
300
```

### Test Case 3

NLOC

```
34
```

# Code

prufer_decode.c

```c
#include <stdio.h>
#include <stdlib.h>

#define MAXN 200005

int code[MAXN];
int degree[MAXN];

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    for (int i = 1; i <= n; i++) degree[i] = 1;
    for (int i = 0; i < n - 2; i++) {
        scanf("%d", &code[i]);
        degree[code[i]]++;
    }

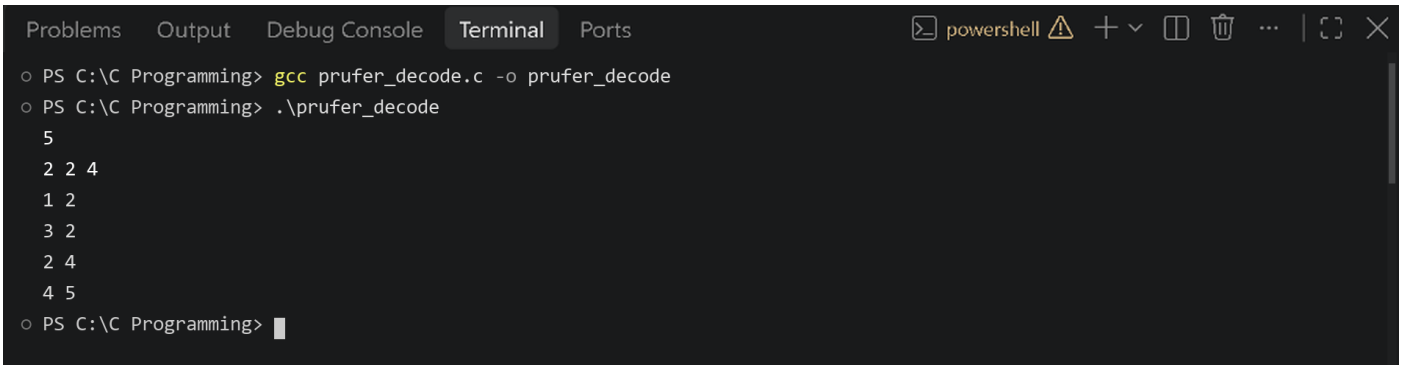
    int ptr = 1;
    while (degree[ptr] != 1) ptr++;
    int leaf = ptr;

    for (int i = 0; i < n - 2; i++) {
        int v = code[i];
        printf("%d %d\n", leaf, v);

        degree[leaf]--;
        degree[v]--;
        if (degree[v] == 1 && v < ptr) {
            leaf = v;
        } else {
            ptr++;
            while (ptr <= n && degree[ptr] != 1) ptr++;
            leaf = ptr;
        }
    }

    int u = 0, v = 0;
    for (int i = 1; i <= n; i++) {
        if (degree[i] == 1) {
            if (u == 0) u = i;
            else v = i;
        }
    }
    printf("%d %d\n", u, v);
    return 0;
}
```

## Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc prufer_decode.c -o prufer_decode
PS C:\C Programming> .\prufer_decode
5
2 2 4
1 2
3 2
2 4
4 5
PS C:\C Programming>